

KINESIS Technical Architecture

Firmware, AI Pipeline, and Application Software

Companion to KINESIS Phase 1 Progress Report

1. Introduction

This document accompanies the KINESIS Phase 1 Progress Report and describes the technical implementation of the system in three layers: the embedded firmware running on the MYOSA wearable, the artificial intelligence pipeline that produces clinical signal from raw sensor input, and the application software that surfaces the results to patients and clinicians. The document is intended as a complete technical reference for reviewers who need to understand how each component is realized in code rather than as an idealized design.

A clarifying note on terminology is warranted at the outset. The KINESIS system is described throughout the public materials as an AI-driven rehabilitation monitoring platform. The intelligence that justifies this description is distributed across several components rather than concentrated in a single model. The pose estimation that produces the body skeleton is a learned model. The orientation fusion that produces the wearable signal is a classical sensor-fusion filter. The joint-angle computation, the rep counter, and the quality scorer are deterministic algorithms operating on the fused signal. The natural-language clinical summary is generated by a large language model. Each component is selected for the part of the problem it solves best, and the report below makes explicit which components are learned and which are not, so that the reviewer can form an accurate picture of where the intelligence lives.

Patient application: <https://kinesis-2d856.web.app/>

Clinician dashboard: <https://kinesis-clinician.web.app/>

2. Firmware Layer

2.1 Platform and Role

The wearable firmware (`kinesis_node.ino`) is a single-file Arduino sketch that runs on the MYOSA motherboard, an ESP32-WROOM-32E platform with onboard USB-to-serial via the CP210x bridge. The firmware turns the MYOSA AccelAndGyro module into a high-rate ground-truth IMU stream consumed by the patient web application. The role of the wearable in the broader system is to provide a single body-segment orientation reference at low latency and high accuracy that the vision pipeline can use as a trust anchor when the camera signal degrades.

2.2 Hardware Configuration

Component	Description and Role
-----------	----------------------

MYOSA motherboard	ESP32-WROOM-32E with USB-C, CP210x USB-serial bridge, on-board regulator, and I2C bus header
AccelAndGyro	6-axis inertial sensor (MPU6050-class) at I2C address 0x69, default 100 kHz bus rate
Power	USB 5V from the patient's laptop during use, with battery-powered operation available via TP4056 and lithium-polymer cell for untethered demos
Mounting	Velcro strap on the active limb segment (e.g. upper arm for elbow exercises)

2.3 Sampling and Sensor Fusion

After initialization, the firmware enters a loop that samples the inertial sensor at 100 Hz and updates a Madgwick attitude and heading reference system filter on every sample. The Madgwick algorithm is a gradient-descent filter that fuses the short-term accuracy of gyroscope integration with the long-term gravity reference of the accelerometer, producing a stable orientation quaternion with low latency. The magnetometer path is unused because the MYOSA inertial sensor is six-axis and not nine-axis. Yaw drift is therefore present, but for the single-limb-segment use case it is tolerable, and the limb orientation around the gravity axis is recovered downstream by the vision pipeline.

2.4 Telemetry Frame Format

Every 50 ms, the latest fused frame is emitted as a single line of JSON over USB serial at 115200 baud. The browser-side parser is intentionally permissive: it extracts whatever keyed numeric fields are present in the line and tolerates NaN and infinity literals, which the standard Arduino floating-point printer emits verbatim when the Madgwick filter briefly destabilizes during boot.

```
{
  "t": 12345,           // millis() at emit time
  "qw": 0.9986,        // fused quaternion components
  "qx": 0.0123,
  "qy": -0.0451,
  "qz": 0.0207,
  "roll": -2.65,       // Euler triple, degrees
  "pitch": 5.18,
  "yaw": 1.23,
  "ax": 0.02, "ay": 0.98, "az": 0.21, // accel, g units
  "gx": 1.2, "gy": -0.4, "gz": 0.1,  // gyro, deg/s
  "romMin": -45.2,     // session statistics, deg
  "romMax": 78.4,
  "romSpan": 123.6,
  "motion": 2.13       // |delta pitch| accumulated since last emit
}
```

2.5 Quaternion Reconstruction

The Madgwick library used in the firmware exposes the fused orientation as Euler angles only. The quaternion fields in the telemetry frame are reconstructed at emit time from the Euler triple using the

standard ZYX rotation composition. The browser side renormalizes the quaternion after parsing to absorb any floating-point rounding accumulated through the conversion.

2.6 Three Independent Timers

The main loop runs three timers in parallel. The first timer drives the 100 Hz sample-and-fuse cycle that updates the orientation quaternion. The second timer drives a 5 Hz status print to the Arduino IDE Serial Monitor that supports developer debugging. The third timer drives the 20 Hz JSON emission that the patient application consumes. The micros and millis arithmetic keeps the three timers phase-stable even when one runs slow, and the inertial read is gated by a ping to the sensor so the loop never blocks on a transaction in progress.

2.7 Auto-Reset on Connect

When the patient application opens the serial port, it pulses the RTS line high then low with an 80 ms delay between the transitions. The CP210x USB-serial bridge wires RTS to the ESP32 EN pin, so the pulse resets the microcontroller. Because DTR is held low throughout, the boot pin stays high and the chip boots into run mode rather than the flash bootloader. The net effect is that every Connect click yields a clean firmware boot into the JSON stream regardless of the chip's previous state, eliminating an entire class of stale-state bugs during patient sessions.

2.8 Failure Modes and Browser Surfacing

Browser symptom	Cause and resolution
Failed to open serial port	Another application holds the port (typically the Arduino IDE Serial Monitor). Close all serial clients.
Port open but no bytes	imu.begin() failed all retries and the firmware halted with FAILED, check sensor stack seating. Reseat the IMU module and reset.
N bytes received but no parseable frames	Boot loop or partial line or non-JSON garbage on the wire. Pulse reset, confirm the boot banner in the Arduino IDE first.
IMU streaming but joint angle frozen	Calibration not done. The IMU has no zero reference until the patient holds rest pose and clicks Calibrate.

3. AI Pipeline

3.1 Design Philosophy

The KINESIS pipeline does not run a single neural network. It runs a sequence of specialized stages, each tuned for the part of the problem it solves best, with the learned components carefully scoped to the steps where learning genuinely outperforms a deterministic algorithm. The reasoning is operational rather than ideological: a stage that can be implemented as an explicit computation is easier to debug, audit, and explain to a clinician than one that hides its decision in millions of weights. Three categories of stage make up the pipeline. Learned components carry the perceptual work that classical methods cannot do well,

specifically the pose estimation from camera input and the natural-language summarization of session data. Signal-processing components carry the smoothing, calibration, and inverse-kinematics work that is mathematically well-defined and benefits from being exact. Deterministic-algorithm components carry the joint-angle, rep-counting, and quality-scoring work that is best expressed as transparent code so a clinician can ask what the score means and receive a precise answer.

3.2 Pipeline Overview

The pipeline is illustrated in Figure 1. The two input streams (camera frames from the patient device and inertial frames from the MYOSA wearable) are processed independently through their respective acquisition stages, then merged at the joint-angle layer through a confidence-weighted fusion. The fused angles drive the rep counter and the muscle-activation tracker, both of which feed into the quality scorer. The quality scorer's output is written to Firestore and triggers a Cloud Function that generates a natural-language session summary using Claude.

Stage-by-stage summary

Stage	Category	What it does
MediaPipe Pose Landmarker	Learned	Returns 33 world-space landmarks per camera frame
One-Euro filter	Signal proc	Smooths per-landmark jitter, especially depth channel
Derived anatomical landmarks	Algorithm	Synthesizes 11 unseen joints from Vitruvian proportions
Bone-length calibration	Algorithm	Locks per-patient limb lengths to median observed
Anatomical inverse kinematics	Signal proc	Preserves joint angles while enforcing rest lengths
Madgwick AHRS (firmware)	Signal proc	Fuses accel and gyro to a stable quaternion at 100 Hz
Vision-IMU fusion	Algorithm	Confidence-weighted blend of camera and inertial estimates
Joint-angle computation	Algorithm	Trigonometry from three-point landmark triples
Rep counter	Algorithm	Hysteresis state machine with anti-jitter guards
Muscle-activation tracker	Algorithm	Inferred from joint kinematics, not measured by EMG
Quality scorer	Algorithm	Weighted average of ROM, smoothness, symmetry, cadence
Session summary	Learned (LLM)	Claude Opus 4.7 writes the clinical narrative

Table 1. KINESIS pipeline stages and the category of computation each one performs.

3.3 MediaPipe Pose Landmarker

The vision side runs Google's BlazePose model via the MediaPipe Tasks API for the browser. Three model variants are deployed and selected at runtime based on the patient device capability. The lite variant runs at approximately 25 frames per second on integrated graphics and is the appropriate choice for older devices and for quick warm-up before a session begins. The full variant runs at approximately 20 frames per second and is the default, offering a balanced trade-off between accuracy and computational load. The heavy variant runs at approximately 12 frames per second and is reserved for clinic-grade recording on workstations with a strong dedicated graphics processor. Each frame returns 33 world-space landmarks in metres along with a per-landmark visibility score in the interval from zero to one. This visibility score is critical to the downstream fusion stage because it provides the per-joint confidence value that determines how heavily the camera contributes relative to the inertial sensor.

3.4 One-Euro Filter

MediaPipe's depth channel is noisy in a way that makes the raw output unsuitable for direct rendering. A naive avatar driven by unfiltered landmarks exhibits a phenomenon that the development team came to refer to as breathing, in which limb lengths flicker by tens of percent each frame. A One-Euro filter is applied to every landmark before any downstream stage consumes the data. The filter is parameterized with a low base cutoff of 1.5 Hz and a beta of 0.03, which removes high-frequency jitter while permitting the cutoff to adapt upward in proportion to the instantaneous derivative of the landmark. The adaptive cutoff preserves fast motion with low latency, so the smoothing is invisible during exercise but the avatar nonetheless feels solid. This stage is signal processing rather than learned, but it is the difference between an avatar that a clinician will use and one that a clinician will dismiss within seconds.

3.5 Derived Anatomical Landmarks

MediaPipe observes only skin-surface landmarks. The real human skeleton has joints that the camera cannot directly see, including the pelvis center, the lumbar and thoracic spine, the sternum, the atlanto-occipital pivot, and the crown of the head. The pipeline synthesizes eleven of these unseen joints on every frame using Vitruvian proportions normalized to the patient's calibrated body height. The pelvis is computed as the midpoint of the left and right hip landmarks. The lumbar landmark sits at thirty-six percent of the distance from pelvis to sternum with a forward offset representing the lordotic curve. The thoracic landmark sits at seventy-two percent of the same distance with a slight backward offset representing the kyphotic curve. The sternum is the midpoint of the left and right shoulders. The neck sits above the sternum along the body-up axis. The head base is the midpoint of the left and right ears, which approximates the true atlanto-occipital pivot. The head top sits above the head base along the body-up axis. Each hand is the centroid of the pinky, index, and thumb landmarks for that side. Each foot is the midpoint of heel and toe. The result is a skeleton with realistic anatomical proportions derived from the patient's own body rather than an arbitrary scale.

3.6 Bone-Length Calibration

For the first approximately twenty-five high-visibility frames of each bone during a session, the calibrator records the observed Euclidean distance between the two endpoint landmarks. After this calibration window, the median of the observed distances is locked as that patient's true rest length for that bone. Body height itself uses the seventy-fifth percentile of the head-top to mid-foot distance, biased toward

fully-extended frames. The locked rest lengths feed the inverse-kinematics stage in the next subsection. The reason this stage exists is that even after One-Euro smoothing, MediaPipe's depth channel produces small frame-to-frame variations in apparent limb length, and these variations would otherwise propagate into the rendered avatar as visible distortion. Calibration converts the noisy magnitude information from MediaPipe into a per-patient constant, leaving only the direction information to drive the avatar.

3.7 Anatomical Inverse Kinematics

For each bone in the parent-to-child hierarchy, the inverse-kinematics pass computes the observed direction from the parent landmark to the child landmark and pins the child at exactly the rest length along that direction, discarding the noisy distance magnitude. The direction follows the patient's real motion frame by frame, the magnitude stays locked at the calibrated value. The result is that every joint angle is preserved exactly while every limb length stays anatomically correct, eliminating the residual distortion that One-Euro smoothing alone cannot remove. This stage is the single most important contributor to the visual quality of the avatar in the patient application and is documented as a deliberate engineering decision rather than as an accident of implementation.

3.8 Joint Angles, Rep Counting, and Muscle Activation

For each tracked joint, the angle is computed from a three-point landmark triple by pure trigonometry. For example, the elbow flexion angle is computed from the shoulder, elbow, and wrist landmarks using the law of cosines on the triangle they form. There is no learned component here and there should not be one, since the geometric definition of a joint angle is exact.

The rep counter is a hysteresis state machine with anti-jitter guards. A per-model confidence threshold (0.5 for lite, 0.6 for full, 0.7 for heavy) suppresses counting on low-quality frames. A direction-change detector with a minimum-travel margin requires the joint to actually traverse the rep before a rep is registered. A maximum-jump cap discards tracking glitches in which the landmark teleports between frames. Per-rep statistics are recorded for the peak angle, trough angle, range of motion, mean angular speed, left-right symmetry, and the inter-rep interval.

Muscle activation is inferred from joint kinematics rather than measured by surface electromyography. When the elbow flexion angle decreases, the biceps is annotated as firing. When the angle increases, the triceps is annotated as firing. The same logic generalizes across every major hinge joint. For each tracked muscle, the relevant joint's signed angular velocity is projected onto a vector representing that muscle's firing direction, clamped to the interval from zero to one, and smoothed by an exponential moving average so that the on-screen muscle bundle pulses rather than flickers. This stage is explicitly documented as an inference rather than a measurement because clinicians who see the activation visualization need to understand what it does and does not represent.

3.9 Vision-IMU Fusion

Where both the vision pipeline and the inertial sensor are tracking the same joint, their outputs are blended through a confidence-weighted average. The vision weight is the per-joint MediaPipe visibility score scaled by a fixed coefficient and clamped to the interval from 0.1 to 0.9. The inertial weight is one minus the vision weight. The fused angle is the convex combination of the two estimates under these weights. When the patient turns sideways and the camera loses sight of the joint, the visibility score drops

below approximately 0.3 and the inertial sensor dominates the fused output. When the patient faces the camera and the visibility score exceeds approximately 0.7, the camera dominates. The transition between the two regimes is continuous rather than switched, so there is no observable jump in the joint-angle signal as the patient moves through occluding poses. This stage is the operational reason the wearable exists at all in Phase 1: it is the trust anchor that prevents the rep counter and the quality scorer from being fooled by a single bad camera frame.

3.10 Quality Scoring

Every 600 ms the pipeline produces a quality score on the current set. The score is a weighted average across four axes presented in Table 2. Each axis is computed from the fused joint-angle stream and the running session statistics.

Axis	Weight	Computed from
Range of motion	40 percent	Ratio of current ROM to target ROM, clamped to the interval from zero to one
Smoothness	25 percent	Root-mean-square jerk, the third derivative of the joint angle; lower is better
Symmetry	20 percent	One minus the normalized absolute difference between left and right ROM
Cadence	15 percent	Penalty for variance greater than two standard deviations from the patient's mean pace

Table 2. *Quality score axes, weights, and signal-processing inputs.*

The four scores combine into a single number on a zero-to-one-hundred scale, which is then mapped into a four-class classification. A score of eighty or above is classified as normal. A score between sixty and eighty is classified as compensatory, indicating asymmetry or off-target range of motion. A score between forty and sixty is classified as guarded, indicating very low movement velocity or a shortened range. A score below forty is classified as abnormal, indicating multiple simultaneous failure modes. The classification is a threshold cascade on the quality score, not a learned classifier, which makes its decisions auditable.

3.11 Session Summary via Claude

When a session finalizes, the patient application updates the corresponding Firestore document. A Cloud Function listens for this state transition and invokes Claude (model identifier claude-opus-4-7) with a structured prompt that includes the per-rep statistics, the patient's recent range-of-motion trajectory, and any active alerts. The model returns a single paragraph of natural-language clinical narrative, which is written back to the session document under the field aiSummary. The clinician reads this narrative on the Notes tab of the patient detail view alongside the raw metrics. An illustrative example of the produced narrative is the following: the patient completed eight of ten prescribed repetitions, range of motion improved from seventy-eight to ninety-one degrees compared to the previous session, two compensatory patterns were flagged in the last set, both involving early shoulder elevation, and the quality trend remains positive. Cached prompts keep the cost per session under one cent.

3.12 What is Deliberately Not Learned

Three components of KINESIS that observers commonly assume are machine learning are documented here explicitly as deterministic algorithms. The joint-angle computation is pure trigonometry on three-point landmark triples; there is no learned model and there should not be one, since the geometric definition is exact. The rep counter is a hysteresis state machine with explicit thresholds; the failure modes of a learned rep counter (mistraining on a particular movement style, opacity around why a rep was missed) are avoided by using transparent rules. The three-dimensional avatar in the patient application is a parametric skeleton built from MediaPipe landmarks and the bone-length calibration; it is not a learned skeleton-rigging model. The pose tracking the user sees in the demo is the combination of MediaPipe, One-Euro, the derived landmarks, the calibration, and the inverse kinematics pass, composed deterministically rather than driven by a single end-to-end model. This is a positive design decision: the resulting system runs fully in the browser on a mid-range phone, gives clinicians a signal they can interrogate, and uses the wearable inertial sensor as the trust anchor that keeps the vision pipeline honest.

4. Application Software

4.1 Repository Layout

The KINESIS codebase is a pnpm and Turborepo monorepo containing two Next.js 14 applications and five shared packages, all client-side, with Firebase providing authentication, storage, and serverless functions. The repository structure is summarized below.

```
kinesis/
  apps/
    patient/      Patient PWA: camera + IMU + 3D mirror
    clinician/    Clinician dashboard: cohort + per-patient
  packages/
    db/           Firestore client, types, queries
    pose/         MediaPipe tracker, angles, rep counter
    imu/          Web Serial IMU pipeline, filters, fusion
    ui/           Shared UI primitives
    ai/           AI summarization client to Claude
  firebase/
    firestore.rules
    firestore.indexes.json
    functions/    Cloud Functions
  kinesis_node.ino  ESP32 firmware at repo root
```

4.2 The Two Deployed Applications

Both applications are static-exported Next.js 14 sites deployed to Firebase Hosting. All authentication and data access occurs client-side through the Firebase JavaScript SDK. There is no server-side rendering and no Cloud Function in the request hot path, which keeps hosting cost negligible and eliminates an entire class of cold-start latency issues.

Patient PWA: <https://kinesis-2d856.web.app/>

Clinician dashboard: <https://kinesis-clinician.web.app/>

4.3 Patient Application Walkthrough

The patient session flow proceeds through a fixed sequence of stages. The patient signs in with email and password through Firebase Authentication. A prescribed exercise is selected from the schedule. Camera permission is requested through an explicit user-gesture click that triggers `getUserMedia`, and the MediaPipe pose model is loaded into a Web Worker for off-main-thread inference. The patient optionally pairs the inertial wearable through the IMU panel, which calls `navigator.serial.requestPort` to enumerate available USB devices, opens the selected port at 115200 baud, pulses RTS to reset the ESP32, and begins consuming JSON frames. At rest pose, the patient clicks Calibrate, which captures the current orientation as the zero reference for the session. The patient clicks Start, which triggers a three-second audio countdown followed by the recording phase.

During the recording phase, the pose tracker emits live frames to the session controller. For each frame, the controller computes joint angles, advances the rep counter, writes a sample to a two-second buffer, and updates the quality score every 600 ms. On a 2 Hz cadence the controller flushes the sample buffer to a Firestore subcollection under the session document and updates the live session header with the current rep count, quality score, and classification. The three-dimensional avatar consumes the same landmark stream as the rep counter through a separate React component, and animates in real time. The patient can hit Record Motion to capture every landmark for the session to a Firestore `motionCaptures` document for later review. When the patient clicks End, the session is finalized with a complete statistics block written to the session document, a pain-feeling modal solicits a 0-to-10 self-report, and the application navigates to the progress screen.

4.4 Clinician Dashboard Walkthrough

The clinician dashboard surfaces patient and cohort state through eight primary routes. The root route presents a cohort view with all patients, their latest session statistics, and a live alert feed. A patients route provides a searchable patient table with status filters. A patient detail route loads the full patient document and subscribes to that patient's sessions and alerts in real time, rendering one of five tabs: an overview tab with the ROM trajectory chart, the quality stack, a side panel of recent activity, the last five sessions, and the AI insight pane; a sessions tab with the complete session history; an exercises tab with live-subscribed prescriptions; a messages tab with a chat thread; and a notes tab with the patient condition and AI session summaries. Separate routes provide alert triage, analytics on quality outliers, an automated weekly PDF report generator, and clinic configuration including invite codes and member management.

A practical implementation note is that the patient detail route uses a search-parameter URL of the form `/patients/view?id=` rather than a Next.js dynamic segment of the form `/patients/[id]`. The dynamic segment form triggers server-side rendering even when every file in the segment is marked as a client component, and Firebase Hosting's framework integration then attempts to bundle the route into a Cloud Function via npm. The bundling step breaks on the pnpm workspace-protocol dependencies used in the monorepo. The search-parameter URL is identical in user experience and keeps the build fully static.

4.5 Shared Packages

Five shared packages provide the core capabilities used by both applications. The db package contains the Firestore client, all type definitions, and the query helpers; the client is initialized with `experimentalForceLongPolling` and an in-memory cache to avoid the well-known internal-assertion crashes

during development hot reloads. The pose package wraps MediaPipe Pose Landmarker, the One-Euro smoothing, and the rep counter into a React component that emits per-joint angles and confidence on one event channel and raw landmarks on a second event channel for the three-dimensional avatar. The imu package wraps the Web Serial transport, the three available orientation filters (Complementary, Madgwick, and DevicePassthrough), and the joint-angle mapper into a React hook that exposes a fused angle to the parent component. The ui package contains pure presentation components with no business logic. The ai package contains a thin Claude API client used by both the clinician's report generator and the Cloud Function that summarizes sessions on finalize.

4.6 Firebase Backend

The Firebase project provides four services. Authentication runs email and password sign-in for both clinicians and patients, with patient sign-up gated by a six-digit clinician invite code. Firestore stores every entity, including patients, sessions, samples, messages, alerts, prescriptions, medications, motion captures, and appointments. Hosting serves both applications as separate sites under the project. Cloud Functions run two scheduled or trigger-based handlers: summariseSession fires when a session is finalized and writes the natural-language summary to the session document, and escalateAlerts runs on a schedule to escalate unacknowledged critical alerts after a twelve-hour timeout.

5. Closing Notes

The KINESIS technical architecture is designed around a deliberate division of labor between learned models, signal-processing filters, and deterministic algorithms. The learned components carry the perceptual work (pose estimation and natural-language summarization) where their advantage over classical methods is decisive. The signal-processing components carry the work (orientation fusion, smoothing, inverse kinematics) where the mathematical structure is well-defined and exactness is preferable to a learned approximation. The deterministic-algorithm components carry the work (joint angles, rep counting, quality scoring, muscle activation inference) where transparency to the clinician is more valuable than a marginal improvement in accuracy. The result is a system that runs entirely on patient and clinician devices, surfaces clinically defensible signal, and uses the MYOSA wearable as the trust anchor that keeps the vision pipeline honest in the conditions where camera-only systems fail.

The document above accompanies the KINESIS Phase 1 Progress Report and is intended as a reference for reviewers who need to confirm how each component is realized in code. The two deployed web applications, the firmware source, and the monorepo containing the AI pipeline and shared packages are the primary engineering deliverables of the Phase 1 milestone.

References

- [1] S. O. H. Madgwick, A. J. L. Harrison, and R. Vaidyanathan, "Estimation of IMU and MARG orientation using a gradient descent algorithm," Proc. IEEE Int. Conf. Rehabilitation Robotics, 2011.
- [2] V. Bazarevsky et al., "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020.
- [3] Google, "MediaPipe Pose Landmarker," MediaPipe Solutions documentation, 2024.

- [4] G. Casiez, N. Roussel, and D. Vogel, "1 Euro filter: a simple speed-based low-pass filter for noisy input in interactive systems," Proc. ACM Conf. Human Factors in Computing Systems, 2012.
- [5] Anthropic, "Claude Opus 4.7 Model Card," 2026.
- [6] InvenSense, "MPU-6000 and MPU-6050 Product Specification," Document PS-MPU-6000A-00, Revision 3.4, 2013.
- [7] Espressif Systems, "ESP32-WROOM-32E Datasheet," Version 1.7, 2023.
- [8] Vercel, "Next.js 14 documentation," 2024.
- [9] Google, "Firebase documentation," 2024.